You said:

Session Start to solve the task-4 problems (**00:00:18**) - 1. Combine two lists index-wise(columns-wise) (**00:02:05**) - 2. Add a new item to the list after a specified item (**00:03:26**) - 3. Update no of items available (**00:04:59**) - 4. Running Sum on the list (**00:06:20**) - 5. You are given a list of integers. You are asked to make a list by running through elements of the list by adding all elements greater and itself. (**00:11:34**) - 6. Find a list of common unique items from two lists. and show in increasing order (**00:13:32**) - 7. Sort a list of alphanumeric strings based on the product value of the numeric character in it. If in any string there is no numeric character take its product value as 1. (**00:18:47**) - 8. Split String of list on K character. (**00:21:19**) - 9. Convert Character Matrix to single String using string comprehension. (**00:23:24**) - 10. Add Space between Potential Words. (**00:30:59**) - 11. Write a program that can perform union operations on 2 lists. (**00:32:47**) - 12. Write a program that can find the max number of each row of a matrix. (**00:34:10**) - 13. Write a list comprehension to print the following matrix (**00:38:36**) - 14. Write a list comprehension that can transpose a given matrix (**00:44:25**) - 15. Write a list comprehension that can flatten a nested list
Explain all concept and code
mind map
coding questions and sol

ChatGPT said:

Python List Practice — Task 4: Complete Concepts, Code, Mind Map & Solutions

This session covers **15 Python coding questions** based on lists, strings, list comprehensions, sets, matrices, and basic problem-solving. I'll explain them in beginner-friendly language with examples, dry runs, expected output, and interview tips.

The goal is not just to copy code. It is to understand the logic so you can solve similar questions independently.

1. Mind map of all concepts

Python Task 4

15 coding problems

↓

A. Lists

1. Combine lists
2. Insert after item
3. Update quantity
4. Running sum
5. Greater-element sums

B. Sets & sorting

6. Common unique items
7. Numeric digit products
11. Union of lists

C. Strings

8. Split on K
9. Join character matrix
10. Add spaces to words

D. Matrices & comprehension

12. Maximum per row
13. Generate a matrix
14. Transpose a matrix
15. Flatten nested lists

Core tools: indexing, loops, conditions, append(), insert(), sum(), set(), sorted(), split(), join(), max(), and list comprehension.

2. Essential concepts before solving

Concept	Meaning	Example
List	Ordered, changeable collection	`[10, 20, 30]`
Index	Position starts at 0	`a[0]` gives `10`
`append(x)`	Add to the end	`a.append(40)`
`insert(i, x)`	Insert at index `i`	`a.insert(1, 15)`
`sum(a)`	Add numeric elements	`sum([2, 3])` → `5`

`set(a)`	Keep unique values	`set([1, 1, 2])`
`sorted(a)`	Return a sorted list	`sorted([3, 1])` → `[1, 3]`
`split()`	Break a string into pieces	`"a-b".split("-")`
`join()`	Combine strings	`"-".join(["a", "b"])`
List comprehension	Build a list in one expression	`[x*x for x in range(4)]`

A **matrix** is a list of lists. For example:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
```

It has 2 rows and 3 columns.

A **list comprehension** follows this general pattern:

```
[expression for item in iterable if condition]
```

Example:

```
squares = [x * x for x in range(1, 5)]
print(squares)
```

Output:

```
[1, 4, 9, 16]
```

Part A — List problems (1–5)

Question 1. Combine two lists index-wise (column-wise)

Problem: Given two lists, combine their elements position by position to create rows or columns.

Example input:

```
names = ["Aamir", "Rahul", "Priya"]
scores = [85, 90, 95]
```

Expected output:

```
[['Aamir', 85], ['Rahul', 90], ['Priya', 95]]
```

Solution

```
names = ["Aamir", "Rahul", "Priya"]
scores = [85, 90, 95]

result = [[name, score] for name, score in zip(names, scores)]

print(result)
```

Output:

```
[['Aamir', 85], ['Rahul', 90], ['Priya', 95]]
```

How it works:

- `zip(names, scores)` pairs elements at matching positions.
- The list comprehension converts each pair into a list.
- `result` stores the combined data.

Dry run:

Iteration	Name	Score	Result so far
1	Aamir	85	`[['Aamir', 85]]`
2	Rahul	90	`[['Aamir', 85], ['Rahul', 90]]`
3	Priya	95	Adds `['Priya', 95]`

Alternative using a loop:

```
result = []

for i in range(len(names)):
    result.append([names[i], scores[i]])

print(result)
```

Note: `zip()` stops when the shorter list ends. If you need to preserve every item from unequal-length lists, consider `itertools.zip_longest()`.

Time complexity: $O(n)$.

Question 2. Add a new item after a specified item

Problem: Insert `"Mango"` immediately after `"Banana"`.

```
fruits = ["Apple", "Banana", "Orange", "Grapes"]
target = "Banana"
new_item = "Mango"

if target in fruits:
    index = fruits.index(target)
    fruits.insert(index + 1, new_item)

print(fruits)
```

Output:

```
['Apple', 'Banana', 'Mango', 'Orange', 'Grapes']
```

Important concepts:

- `target in fruits` checks whether the item exists.
- `fruits.index(target)` returns its first index.
- `index + 1` specifies the position after it.
- `insert()` adds the new item without replacing an existing one.

Dry run:

```
Original:       Apple, Banana, Orange, Grapes
Banana index:   1
Insert index:   1 + 1 = 2
Final:          Apple, Banana, Mango, Orange, Grapes
```

If the target does not exist, the list remains unchanged.

Interview variation: Insert an item before a target by using `fruits.insert(index, new_item)` instead.

Question 3. Update the number of items available

Problem: Update the quantity of a product in a list of inventory records.

```
products = [
    ["Laptop", 10],
    ["Mouse", 25],
    ["Keyboard", 15]
]

product_name = "Mouse"
new_quantity = 40

for product in products:
    if product[0] == product_name:
        product[1] = new_quantity
        break

print(products)
```

Output:

```
[['Laptop', 10], ['Mouse', 40], ['Keyboard', 15]]
```

Logic:

- `product[0]` is the product name.

- `product[1]` is the available quantity.
- When the product name matches, update its quantity.
- `break` stops the loop after the first match.

Dry run:

Product	Old quantity	Action
Laptop	10	No change
Mouse	25	Update to 40
Keyboard	15	Not checked after `break`

This is an example of **updating nested lists**.

Question 4. Running sum of a list

Problem: Return a list in which every element contains the sum of all elements up to that position.

Input:

```
numbers = [1, 2, 3, 4, 5]
```

Output:

```
[1, 3, 6, 10, 15]
```

Solution 1: Using a loop

```
numbers = [1, 2, 3, 4, 5]

running_sum = []
total = 0

for num in numbers:
    total += num
    running_sum.append(total)
```

```
print(running_sum)
```

Dry run

Current number	Calculation	Running total	Output list
1	`0 + 1`	1	`[1]`
2	`1 + 2`	3	`[1, 3]`
3	`3 + 3`	6	`[1, 3, 6]`
4	`6 + 4`	10	`[1, 3, 6, 10]`
5	`10 + 5`	15	`[1, 3, 6, 10, 15]`

Solution 2: Using `itertools.accumulate()`

```
from itertools import accumulate

numbers = [1, 2, 3, 4, 5]

print(list(accumulate(numbers)))
```

Output:

```
[1, 3, 6, 10, 15]
```

Key difference: `sum(numbers)` gives only the final sum, while a running sum records every intermediate total.

Time complexity: $O(n)$.

Question 5. Sum each element with all elements greater than or equal to it

Problem interpretation: For each number, add that number and every number greater than it. This interpretation treats “greater and itself” as greater than or equal to the current number.

Input:

```
numbers = [1, 3, 2, 4]
```

Expected output:

```
[10, 7, 9, 4]
```

Why?

- For `1`: $1 + 2 + 3 + 4 = 10$
- For `3`: $3 + 4 = 7$
- For `2`: $2 + 3 + 4 = 9$
- For `4`: $4 = 4$

Solution

```
numbers = [1, 3, 2, 4]

result = []

for num in numbers:
    total = sum(x for x in numbers if x >= num)
    result.append(total)

print(result)
```

Output:

```
[10, 7, 9, 4]
```

Understand the condition

```
x >= num
```

This means `x` must be greater than or equal to the current number.

For `num = 2`, the condition selects `3`, `2`, and `4`, so the sum is `9`.

Complexity: $O(n^2)$, because the list is scanned for each element.

If the original problem means *strictly greater* elements, replace `>=` with `>`. If it instead means the sum of the current element and all elements to its right, the solution will be different.

Part B — Sets, sorting and unions (6–7, 11)

Question 6. Find common unique items from two lists in increasing order

Problem: Find elements present in both lists, remove duplicates, and sort them.

```
list1 = [5, 1, 2, 2, 3, 8]
list2 = [2, 3, 3, 4, 5, 9]

result = sorted(set(list1) & set(list2))

print(result)
```

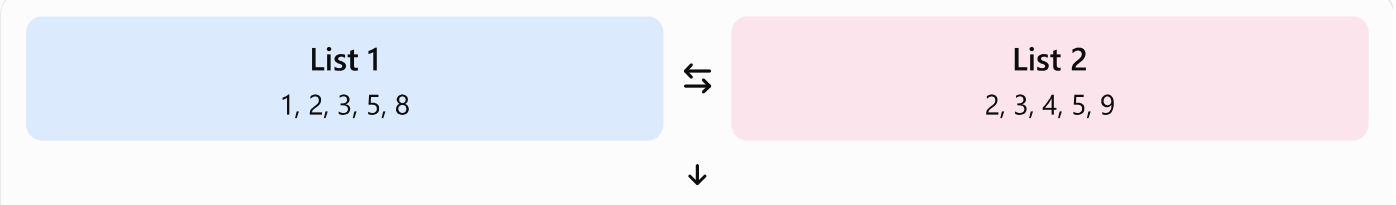
Output:

```
[2, 3, 5]
```

Concepts:

- `set(list1)` removes duplicates.
- `&` finds the intersection of two sets.
- `sorted()` arranges the result in ascending order.

Visual explanation:



Intersection

2, 3, 5

Alternative without sets:

```
result = sorted(set(x for x in list1 if x in list2))
print(result)
```

For large lists, using sets is generally more efficient than repeatedly searching a list.

Question 7. Sort alphanumeric strings by the product of their numeric digits

Problem: Sort strings based on the product of the digits in each string. If there are no digits, use a product value of `1`.

Example:

```
items = ["a23", "b12", "hello", "c34", "d5"]
```

Digit products:

String	Digits	Product
`"a23"`	2, 3	6
`"b12"`	1, 2	2
`"hello"`	None	1
`"c34"`	3, 4	12
`"d5"`	5	5

Expected sorted output:

```
['hello', 'b12', 'd5', 'a23', 'c34']
```

Solution

```
from math import prod

items = ["a23", "b12", "hello", "c34", "d5"]

def digit_product(text):
    digits = [int(ch) for ch in text if ch.isdigit()]
    return prod(digits) if digits else 1

result = sorted(items, key=digit_product)

print(result)
```

How it works:

1. `ch.isdigit()` identifies numeric characters.
2. `int(ch)` converts each digit into an integer.
3. `prod(digits)` multiplies the digits together.
4. If no digits exist, return `1`.
5. `sorted(..., key=digit_product)` sorts strings by their calculated products.

Example calculation:

$$2 \times 3 = 6$$

For `"a23"`, the sorting key is `6`.

Interview tip: Python's sort is stable. If two strings have the same product, their original relative order is preserved.

Question 11. Perform the union of two lists

Problem: Combine two lists and retain only unique elements.

```
list1 = [1, 2, 3, 4]
list2 = [3, 4, 5, 6]

result = list(set(list1) | set(list2))
```

```
print(result)
```

The result contains:

```
1, 2, 3, 4, 5, 6
```

The exact order of the displayed values is not guaranteed when converting a set to a list.

To get a predictable ascending order:

```
result = sorted(set(list1) | set(list2))
print(result)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Union versus intersection:

Operation	Meaning	Operator
Union	All unique elements from both	`set(a) | set(b)`
Intersection	Only common elements	`set(a) & set(b)`
Difference	In first set, not second	`set(a) - set(b)`
Symmetric difference	In either set but not both	`set(a) ^ set(b)`

If you need to preserve the first-seen order instead of sorting:

```
result = list(dict.fromkeys(list1 + list2))
print(result)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Part C — String problems (8–10)

Question 8. Split a string in a list on the character `K`

Problem: Split each string wherever the letter `"K"` occurs.

Example:

```
words = ["ABKCD", "KPYTHONK", "HELLO"]
```

Solution

```
words = ["ABKCD", "KPYTHONK", "HELLO"]

result = [word.split("K") for word in words]

print(result)
```

Output:

```
[['AB', 'CD'], ['', 'PYTHON', ''], ['HELLO']]
```

Explanation:

- `"ABKCD".split("K")` gives `['AB', 'CD']`.
- A separator at the beginning or end can produce an empty string.
- If the separator does not occur, the original string stays as one item in the resulting sublist.

If you want to split on lowercase and uppercase `k` alike, normalize the text first:

```
result = [word.upper().split("K") for word in words]
```

Remember: `split()` divides a string; it does not remove the entire word or automatically flatten a nested list.

Question 9. Convert a character matrix into one string using comprehension

Problem: Combine the characters stored in a nested list into one string.

Input:

```
matrix = [
    ['P', 'y', 't'],
    ['h', 'o', 'n']
]
```

Expected output:

```
Python
```

Solution using nested comprehension

```
matrix = [
    ['P', 'y', 't'],
    ['h', 'o', 'n']
]

result = ''.join(char for row in matrix for char in row)

print(result)
```

Output:

```
Python
```

Understand the nested comprehension:

```
char for row in matrix for char in row
```

Read it as:

1. Take each `row` from `matrix`.
2. Take each `char` from that row.
3. Pass all characters to `''.join(...)`.
4. Combine them with no separator.

Dry run:

Row	Characters added	String so far
`['P','y','t']`	P, y, t	`Pyt`
`['h','o','n']`	h, o, n	`Python`

Alternative:

```
result = ''.join(''.join(row) for row in matrix)
print(result)
```

Both approaches produce the same result.

Question 10. Add spaces between potential words

Problem: Convert a camel-case string into readable words.

Example:

```
PythonIsEasyToLearn
```

Expected output:

```
Python Is Easy To Learn
```

This problem is commonly solved using regular expressions, because uppercase letters mark the beginnings of words.

Solution

```
import re

text = "PythonIsEasyToLearn"

result = re.sub(r'(?<!^)([A-Z])', r' \1', text)

print(result)
```

Output:

```
Python Is Easy To Learn
```

Explanation:

- `(?<!^)` prevents inserting a space before the first character.
- `([A-Z])` identifies an uppercase English letter.
- `r' \1'` inserts a space before the matched uppercase letter.

Another example:

```
text = "AamirHussainAzad"
print(re.sub(r'(?<!^)([A-Z])', r' \1', text))
```

Output:

```
Aamir Hussain Azad
```

Important limitation: This simple pattern does not perfectly handle every acronym. For example, `"AIModelAPI"` may need more advanced rules if you want `"AI Model API"`.

Part D — Matrix and list-comprehension problems (12–15)

Question 12. Find the maximum number in each row of a matrix

Problem: Find the largest number in every row.

Input:

```
matrix = [
    [1, 5, 3],
    [9, 2, 7],
    [4, 8, 6]
]
```

Expected output:

```
[5, 9, 8]
```

Solution using list comprehension

```
matrix = [
    [1, 5, 3],
    [9, 2, 7],
    [4, 8, 6]
]

result = [max(row) for row in matrix]

print(result)
```

Output:

```
[5, 9, 8]
```

Dry run:

Row	`max(row)`

`[1, 5, 3]`	5
`[9, 2, 7]`	9
`[4, 8, 6]`	8

Solution using a loop

```
result = []

for row in matrix:
    largest = max(row)
    result.append(largest)

print(result)
```

Complexity: $O(r \times c)$, where r is the number of rows and c is the number of elements per row.

Question 13. Write a list comprehension to generate a matrix

The exact target matrix is not included in your message, so here is a representative interview exercise: generate a 3×4 matrix containing consecutive integers from 1 to 12.

Expected matrix:

```
[
    [1,  2,  3,  4],
    [5,  6,  7,  8],
    [9, 10, 11, 12]
]
```

Solution

```
rows = 3
cols = 4

matrix = [
    [i * cols + j + 1 for j in range(cols)]
    for i in range(rows)
```

```
]

for row in matrix:
    print(row)
```

Output:

```
[1, 2, 3, 4]
[5, 6, 7, 8]
[9, 10, 11, 12]
```

Understand the formula:

```
i * cols + j + 1
```

- `i` is the row index.
- `j` is the column index.
- `cols` is the number of columns.
- `+ 1` starts the values at 1 instead of 0.

For row `i = 1`, the values are:

- `1 * 4 + 0 + 1 = 5`
- `1 * 4 + 1 + 1 = 6`
- `1 * 4 + 2 + 1 = 7`
- `1 * 4 + 3 + 1 = 8`

Another common interview variation: Create a matrix in which every cell contains the sum of its row and column indices.

```
matrix = [[i + j for j in range(4)] for i in range(3)]
print(matrix)
```

Output:

```
[[0, 1, 2, 3], [1, 2, 3, 4], [2, 3, 4, 5]]
```

The key is to identify the pattern in the required output before writing the comprehension.

Question 14. Transpose a matrix using list comprehension

Problem: Convert rows into columns and columns into rows.

Original matrix (2 rows × 3 columns):

```
1  2  3
4  5  6
```

Transposed matrix (3 rows × 2 columns):

```
1  4
2  5
3  6
```

Solution

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]

transpose = [
    [matrix[i][j] for i in range(len(matrix))]
    for j in range(len(matrix[0]))
]

print(transpose)
```

Output:

```
[[1, 4], [2, 5], [3, 6]]
```

How the indices work

- `len(matrix)` is the number of rows: 2.
- `len(matrix[0])` is the number of columns: 3.
- The outer loop iterates over columns.
- The inner loop collects that column's elements from every row.

Dry run:

Column index `j`	Values collected	New row
0	`matrix[0][0]`, `matrix[1][0]`	`[1, 4]`
1	`matrix[0][1]`, `matrix[1][1]`	`[2, 5]`
2	`matrix[0][2]`, `matrix[1][2]`	`[3, 6]`

Shorter Python solution

```
transpose = [list(row) for row in zip(*matrix)]
print(transpose)
```

Here, `zip(*matrix)` unpacks the rows and groups the elements column-wise.

Question 15. Flatten a nested list using list comprehension

Problem: Convert a list of lists into a single list.

Input:

```
nested = [
    [1, 2, 3],
    [4, 5],
    [6, 7, 8]
]
```

Expected output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Solution

```
nested = [
    [1, 2, 3],
    [4, 5],
    [6, 7, 8]
]

flat = [item for row in nested for item in row]

print(flat)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Read the expression from left to right:

```
item for row in nested for item in row
```

1. Visit each row.
2. Visit every item in that row.
3. Add each item to the new list.

Dry run

Row	Items collected	Flattened list
`[1, 2, 3]`	1, 2, 3	`[1, 2, 3]`
`[4, 5]`	4, 5	`[1, 2, 3, 4, 5]`
`[6, 7, 8]`	6, 7, 8	`[1, 2, 3, 4, 5, 6, 7, 8]`

Important: This approach flattens one level only. It does not recursively flatten lists nested to arbitrary depths.

3. Quick revision sheet — all 15 solutions

Use this table to revise the important Python patterns before an interview.

Q.	Problem	Main technique
1	Combine lists index-wise	`zip()`
2	Insert after an item	`index()` + `insert()`
3	Update item quantity	Nested list + loop
4	Running sum	Accumulator
5	Sum greater elements and itself	Condition + `sum()`
6	Common unique elements	`set()` + `&` + `sorted()`
7	Sort by digit product	`isdigit()` + `prod()` + `sorted()`
8	Split on K	`split()`
9	Character matrix to string	`join()` + nested comprehension
10	Add spaces between words	`re.sub()`
11	Union of lists	`set()` + `|`
12	Maximum in each row	`max()`
13	Generate a matrix	Nested comprehension
14	Transpose a matrix	Reverse row/column indexing
15	Flatten a nested list	Nested comprehension

4. Practice questions — test your understanding

Try these yourself before opening the solutions. These are new variations on the concepts covered above.

Coding practice

0 of 6 revealed

Solve each problem on paper or in VS Code first, then reveal the solution.

1. Combine two lists index-wise

Input

a = [10, 20, 30]

b = ['A', 'B', 'C']

Reveal solution

2. Insert 99 after 20

Input

numbers = [10, 20, 30, 40]

Reveal solution

3. Calculate a running sum

Input

numbers = [2, 4, 1, 3]

Reveal solution

4. Find sorted common unique values

Input

a = [7, 2, 2, 5]

b = [5, 7, 9, 5]

Reveal solution

5. Flatten a nested list

Input

a = [[1, 2], [3], [4, 5]]

Reveal solution

6. Find the maximum in each row

Input

a = [[3, 9], [8, 2], [4, 7]]

Reveal solution

Reveal all solutions

5. Common mistakes to avoid

1. **Confusing `append()` and `extend()`:** `append([3, 4])` adds one nested list; `extend([3, 4])` adds two individual elements.
2. **Forgetting that indexing starts at zero:** The item after index `i` is inserted at `i + 1`.
3. **Using `set()` when order matters:** A set does not guarantee the original list order.
4. **Confusing running sum with total sum:** Running sum returns multiple intermediate totals.
5. **Reversing the matrix indices:** Transposition requires iterating over the original columns first.
6. **Misreading nested comprehensions:** `[x for row in matrix for x in row]` loops over rows first and then items.
7. **Ignoring edge cases:** Test empty lists, duplicate values, missing targets, and strings without digits.

6. Your recommended learning order

Step 1 — Lists and loops

Questions 1–5. Practise indexing, updating, insertion, accumulators, and conditions.

Step 2 — Sets and sorting

Questions 6, 7 and 11. Learn uniqueness, intersection, union, and custom sorting keys.

Step 3 — Strings

Questions 8–10. Practise splitting, joining, character processing, and regular expressions.

Step 4 — Matrices and comprehensions

Questions 12–15. Focus on nested loops, row/column indexing, transpose, and flattening.

Final challenge: Solve all 15 questions again without looking at the solutions. For every question, explain the input, expected output, logic, and time complexity in your own words. That is how you turn these exercises into real coding and interview skills.